

Projet d'architecture

Simulation d'une machine virtuelle

- I) Assembleur et langage machine - Traduction
- II) Simulation du programme - Execution

I) Assembleur

5ef4 <-> Add a,b

Code machine et son éivalent symbolique

5 bits	1 bit	5 bits	5 bits	16 bits
Code opératoire	Imm.	Dest (rd)	Src1 (rn)	Src2 (rm ou #n)

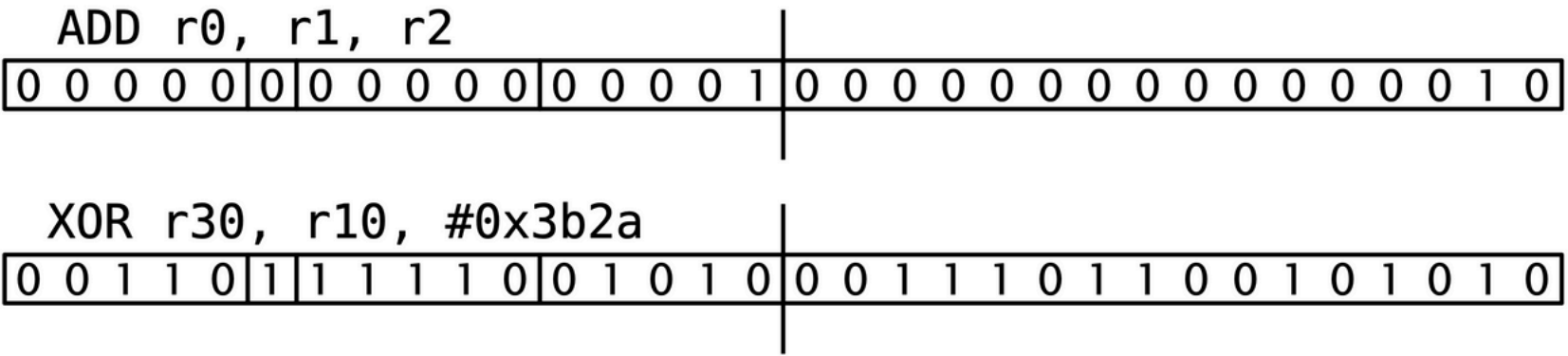
Format général d’une instruction

```
typedef struct{
    int op;
    int dest;
    int src1;
    int imm;
    int src2;
}t_instruction;
```

Structure d’une instruction

0	add rd, rn, S	$rd \leftarrow rn + S$
1	sub rd, rn, S	$rd \leftarrow rn - S$
2	mul rd, rn, S	$rd \leftarrow rn \times S$ (on ne prend que les 8 bits de poids faible de rn et S)
3	div rd, rn, S	$rd \leftarrow rn / S$ (division entière)
4	and rd, rn, S	$rd \leftarrow rn \& S$
5	or rd, rn, S	$rd \leftarrow rn S$
6	xor rd, rn, S	$rd \leftarrow rn \wedge S$
7	shl rd, rn, S	$rd \leftarrow rn$ décalé S fois à gauche (ou à droite si S est négatif), on remplace les nouveaux bits par 0.

Instructions arithmétiques et logiques

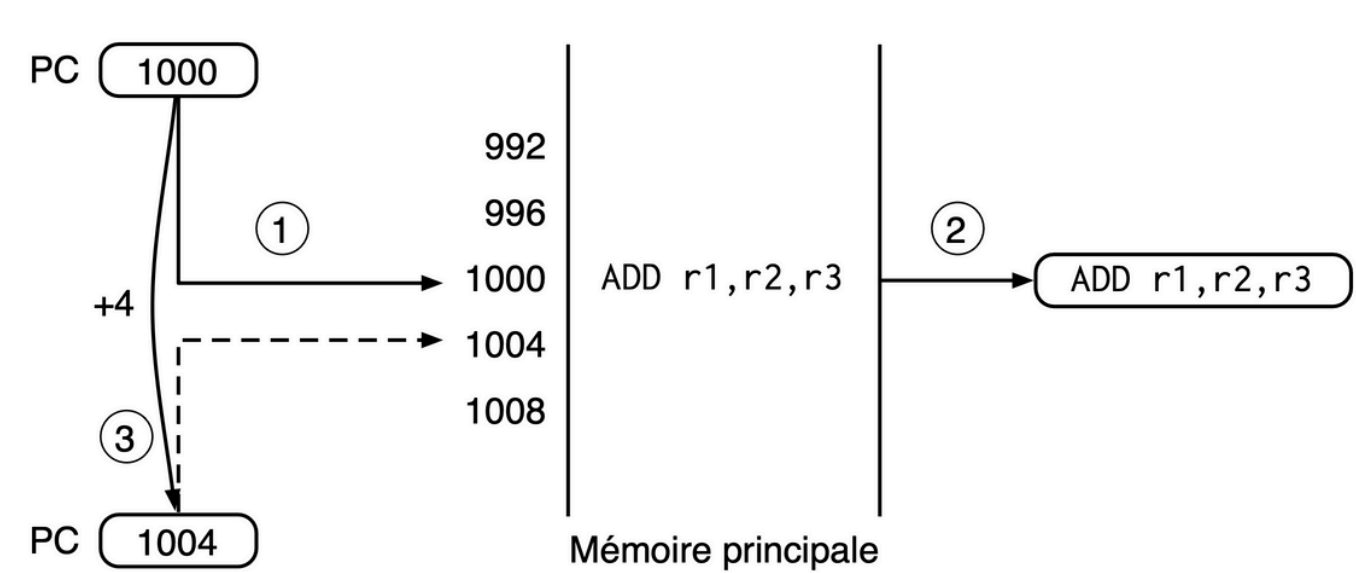


Exemples sur l’addition et le “ou exclusif”

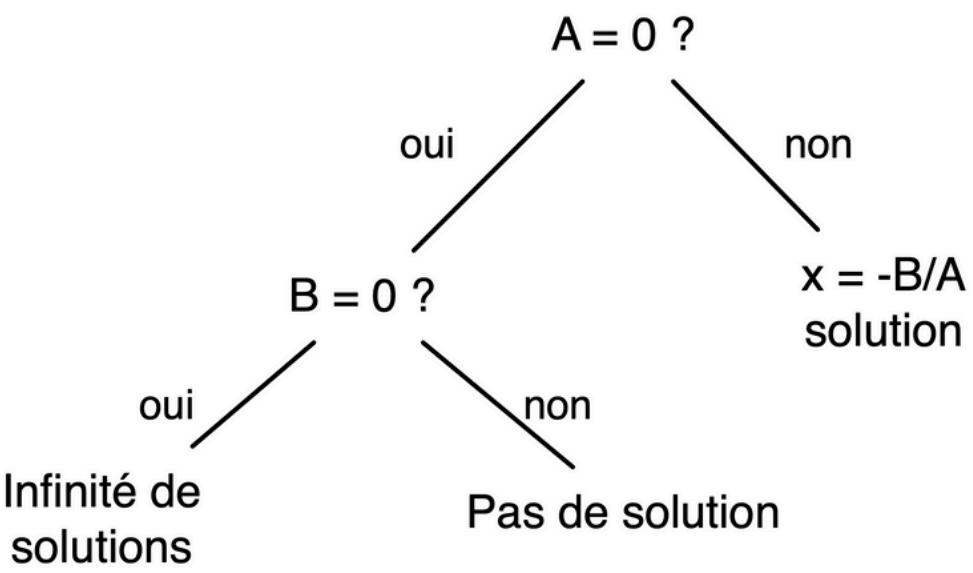
II) Simulation

Adresse	étiquette	mnémonique	Code num.
0x0000	ici:	in r1	D8400000
0x0004		jzs fin	A8010014
0x0008		sub r1, r0, r1	08400001
0x000c		out r1	E0400000
0x0010		jmp ici	A0010000
0x0014		fin: hlt	F8000000

Programme qui affiche l'inverse d'un nombre tant que l'utilisateur ne rentre pas la valeur 0



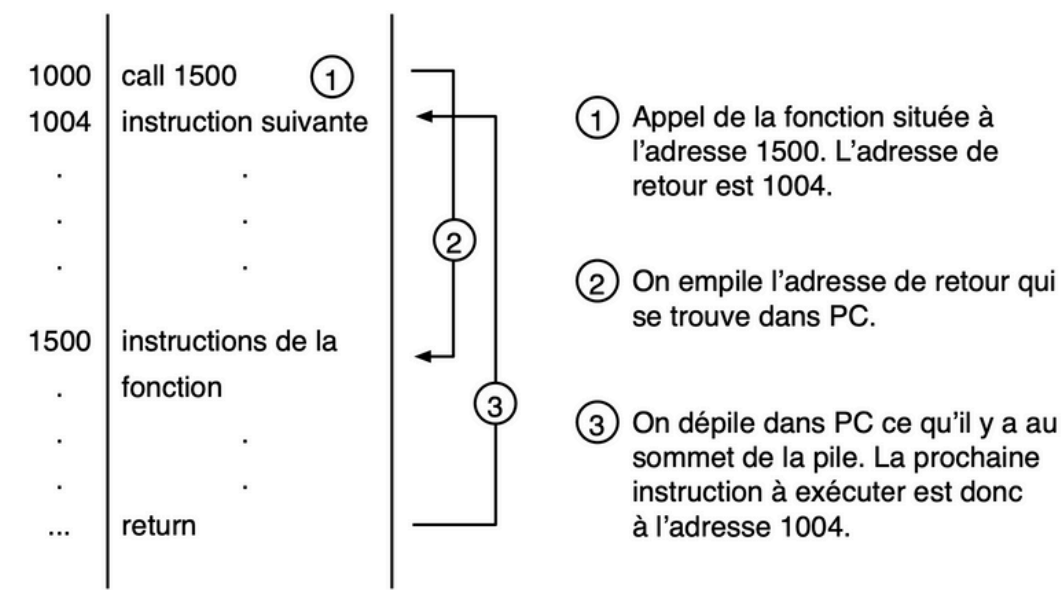
Incrémentation du compteur ordinal (PC)



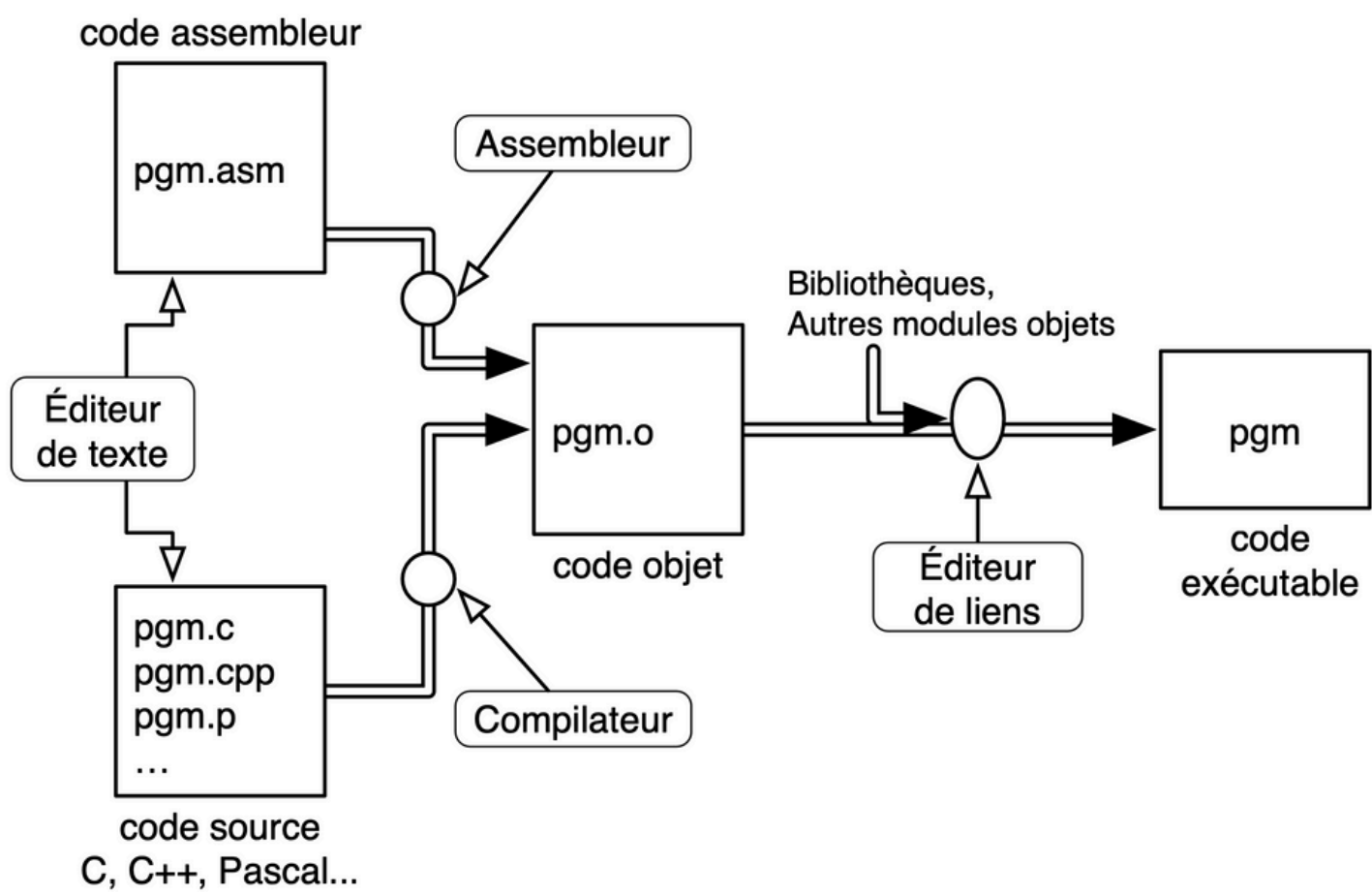
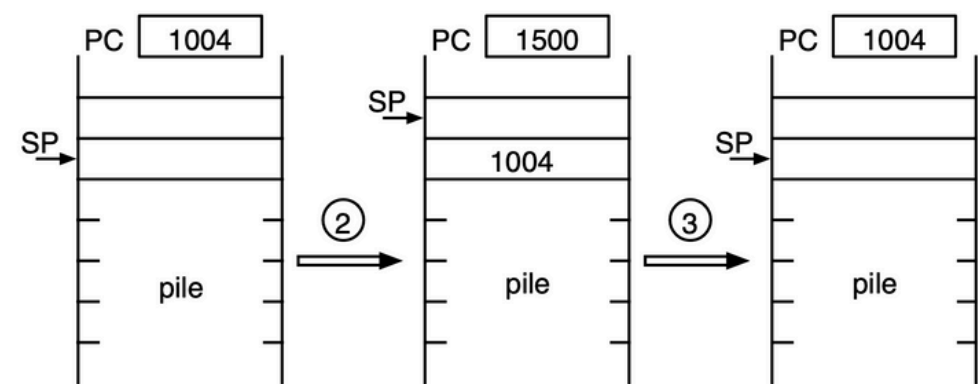
Organigramme de l'équation $Ax + B = 0$

20	jmp S	PC ← S
21	jzs S	PC ← S, si Z=1 (<i>Jump if Z Set</i>)
22	jzc S	PC ← S, si Z=0 (<i>Jump if Z Clear</i>)
23	jcs S	PC ← S, si C=1 (<i>Jump if C Set</i>)
24	jcc S	PC ← S, si C=0 (<i>Jump if C Clear</i>)
25	jns S	PC ← S, si N=1 (<i>Jump if N Set</i>)
26	jnc S	PC ← S, si N=0 (<i>Jump if N Clear</i>)

Instructions de sauts



Appel d'une fonction



Cycle d'un programme compilé